

CS 440/540 Spring 2026

Assignment #2: Recursive Descent Parsing

Due date on Canvas

The goal of this assignment is to use Lex (flex, jflex) write a recursive descent parser in C or Java that will parse a Rusty program. **This is to be your own individual effort.**

To implement this, there are a number of steps. Be sure each step works before going on to the next step.

1. You can start with the lexer you created for assignment one or create a new one.
Remember that the job of a lexer is to capture and return the lexical elements in the grammar and to discard all white space. You will see an example of how to do this in the lecture and in the online examples. NOTE: The lexical elements described below are a subset of what will be used in the later assignments.
2. Convert the Rusty grammar (end of this document) to an LL(1) grammar. For the most part, this means removing left recursion but there may be some common prefixes as well. NOTE: The given grammar is a subset of the grammar we will use in assignments 3 and 4.
3. Write the recursive descent parser for the LL grammar of step 2 - have it use the lexical elements returned by your lexer of step 1. Program input should be from stdin. The output from this should either be:
 - o Input excepted with a count of the number of input lines OR
 - o Syntax error found with the line number of the error.You know this step is working if you can parse all the given input files, as well as input files you create.
4. Be sure to test some input that is incorrect. At the point of the error, print out 'Syntax Error found on line xx', where xx is the line where the error occurred. Terminate your parser at this point.

Lexical elements

The lexical elements are the capitalized elements in the grammar are duplicated here. You can use different token names if you like:

- Keywords: **else** (ELSE_T), **fn** (FN_T), **if** (IF_T), **i32** (I32_T), **let** (LET_T), **mut** (MUT_T), **println** (PRINTLN_T), **then** (THEN_T)
- ID – Alphabetic character followed by zero or more alpha-numeric characters
- INT_LIT are non-negative integer values
- Operators: **,** **+** **-** ***** **/** **(** **)** **{** **}** **;** **=** **:**
- Operator **->** is RET_SYM
- Comments – the user can include comments in the source code. A comment starts with a **!** symbol to the end of the current line.

Grammar

```
rusty → functions
functions → functions function
           | function
function → FN_T ID ( opt_parameter ) opt_return { statements }
opt_parameter → params
              | ε
params → params , param
       | param
param → ID : I32_T
opt_return → RET_SYM I32_T
           | ε
statements → statements statement
           | statement
statement → LET_T ID = expr ;
           | LET_T MUT_T ID = expr ;
           | expr ;
           | PRINT_T LP expr RP ;
           | IF_T expr THEN_T statement
           | IF_T expr THEN_T statement ELSE_T statement
           | { statements }
opt_actual → actuals
           | ε
actuals → actuals , expr
        | expr
expr → expr + term | expr - term | term
term → term * factor | term / factor | factor
factor → ID | ID ( opt_actual ) | INT_LIT | ( expr )
```

Submitting

Submit a TAR or Zip file on Gradscope/Canvas.

- If you are submitted C, your program must compile using flex and gcc (using no library except -ll). We will run your program: `a.out < input`
- If you are submitting Java, your program must compile using the jar file given on blackboard. `java -jar ~white/JFlex.jar program1.jflex`
`javac program1.java`
We will run your code similarly to C: `java program1 < input`
- If there is anything else a grader would need to generate or run your code, put that information in the comment part of the submission.